

环境搭建和 Tools (WSL2 + Ubuntu 24 适配)

- 学号 2351232 姓名 魏义乾

本说明文档介绍了如何在 Windows 下通过 WSL2 安装 Ubuntu 24, 并在本地 VS Code 中进行 xv6-labs-2024 的开发和调试。内容适配本仓库 ([qiankun25/xv6-labs2024](#)) , 并补充了 xv6-labs 官方实验 Git 管理规范与实践。

目录

- 环境搭建和 Tools (WSL2 + Ubuntu 24 适配)
 - 目录
 - 一、在 Windows 上安装 WSL2 和 Ubuntu 24
 - 二、本地 VS Code 连接 WSL2 子系统
 - 1、安装 Remote - WSL 插件
 - 2、直接在 VS Code 中连接 Ubuntu 子系统
 - 三、在 Ubuntu (WSL2) 中安装实验所需工具
 - 四、验证环境
 - 五、Git 管理与实验分支规范
 - 1. 基本配置
 - 2. 获取 xv6-labs2024 仓库
 - 3. 分支管理建议
 - 4. 常用 Git 命令与实践
 - 5. 变基、冲突与提交整理
 - 6. 参考资源
-

一、在 Windows 上安装 WSL2 和 Ubuntu 24

1. 启用 WSL2 功能

- 打开 PowerShell (以管理员身份) 输入:

```
ws1 --install
```

- 按提示自动安装所需的子系统和虚拟机平台, 重启电脑。

2. 安装 Ubuntu 24 子系统

- 在 Microsoft Store 搜索 "Ubuntu 24.04 LTS" 并安装。
- 安装完毕后, 首次启动 Ubuntu, 设置用户名和密码。

3. 确认 WSL2 版本

- 在 PowerShell 输入:

```
wsl --list --verbose
```

- 若版本不是 2，可用以下命令转换：

```
wsl --set-version Ubuntu-24.04 2
```

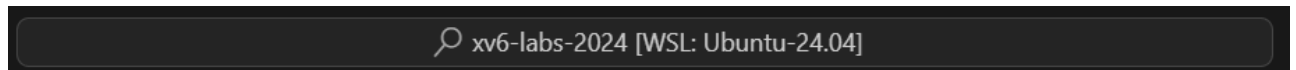
二、本地 VS Code 连接 WSL2 子系统

1、安装 Remote - WSL 插件

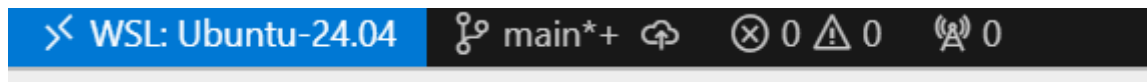
- 打开 Windows 下的 VS Code，进入扩展商店，搜索并安装 **Remote - WSL** 插件。

2、直接在 VS Code 中连接 Ubuntu 子系统

- 安装好插件后，状态栏会显示 **>> WSL: Ubuntu-24.04**，点击即可进入 WSL 环境。



- 也可按 **Ctrl+Shift+P**，输入 **Remote-WSL: New Window**，选择你的 Ubuntu 子系统。
- 此时，所有 VS Code 的终端、文件操作、调试都在 Ubuntu 子系统中进行。



三、在 Ubuntu (WSL2) 中安装实验所需工具

1. 更新系统包

```
sudo apt-get update && sudo apt-get upgrade
```

2. 安装核心工具

```
sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

- 其中：
 - **qemu-system-misc**：RISC-V 架构模拟器（建议版本 7.2 及以上）。
 - **gcc-riscv64-linux-gnu**：RISC-V 交叉编译器。
 - **gdb-multiarch**：多架构调试器。

3. (可选) 安装 C/C++ 插件

- 在 VS Code 的扩展商店搜索并安装 **C/C++**，用于代码高亮和调试。

四、验证环境

1. 验证 QEMU 版本 (需 7.2+)

```
qemu-system-riscv64 --version
```

2. 验证 RISC-V GCC

```
riscv64-linux-gnu-gcc --version
```

3. 验证 gdb-multiarch

```
gdb-multiarch --version
```

4. 验证 Git

```
git --version
```

5. (可选) 克隆本项目并测试 make

```
git clone https://github.com/qiankun25/xv6-labs2024.git  
cd xv6-labs2024  
make qemu
```

五、Git 管理与实验分支规范

1. 基本配置

1. 设置用户名与邮箱 (仅首次)

```
git config --global user.name "你的姓名"  
git config --global user.email "你的邮箱"
```

2. 查看当前配置

```
git config --list
```

2. 获取 xv6-labs2024 仓库

1. 克隆仓库

```
git clone https://github.com/qiankun25/xv6-labs2024.git
cd xv6-labs2024
```

2. 查看所有分支

```
git branch -a
```

3. 分支管理建议

- 每个实验一个分支

官方建议每个实验都新建独立分支。例如：

```
git checkout -b util          # Lab1: UNIX工具
git checkout -b syscall       # Lab2: 系统调用
git checkout -b pgtbl         # Lab3: 页表
# 依此类推
```

分支命名可与官方实验文档保持一致。

- 切换分支时注意保存未提交内容

切换前请确保工作区干净（使用 `git status` 检查；如需暂存可用 `git stash`）。

- 多人协作时，避免在 `main/master` 分支直接开发。

4. 常用 Git 命令与实践

- 查看状态和日志

```
git status
git log --oneline --graph --all
```

- 添加与提交

```
git add <filename>    # 添加单个文件
git add .              # 添加所有文件
git commit -m "简明提交说明"
```

- 推送到远端

```
git push origin <branch>
```

- **从远端拉取更新**

```
git pull
```

- **切换分支**

```
git checkout <branch>
```

- **创建新分支**

```
git checkout -b <new-branch>
```

- **合并分支 (如需)**

```
git checkout main  
git merge <feature-branch>
```

- **解决冲突**

- 按提示编辑冲突文件，修正后：

```
git add <conflicted-file>  
git commit
```

5. 变基、冲突与提交整理

- **变基用于整理历史，合并少量相关提交**

```
git rebase -i HEAD~3
```

按需 squash/reword。

- **撤销错误操作**

- 撤销未 add 的修改：

```
git checkout -- <file>
```

- 撤销已 add 但未 commit 的内容：

```
git reset HEAD <file>
```

- **建议每个实验提交若干次，保持每个 commit 逻辑清晰，不宜一次大提交。**

6. 参考资源

- [xv6-labs 官方 Git 管理建议](#)
- [廖雪峰 Git 教程](#)
- [Pro Git 中文版](#)
- [MIT xv6-labs 官方仓库](#)

这样，就可以在 Windows 下使用 WSL2 + Ubuntu 24 + VS Code + Git 高效完成 xv6-labs2024 的全部实验开发、管理与提交。